

第12章 装饰器与函数式编程

装饰器本质 · wraps · 带参数装饰器 · contextmanager · lru_cache

本章目标:讲透装饰器:从"函数是对象"的本质出发,到 @ 语法糖、functools.wraps、带参数装饰器、类装饰器,再到 lru_cache/contextmanager 等内置宝库,最后手写生产级计时、重试、缓存装饰器。学完你能把日志、鉴权、超时、重试这类横切逻辑从业务代码里彻底蒸发。

直觉起点:C 里"给所有函数加日志"要改每个函数,或造宏、函数指针表;Python 的装饰器把"包装函数"变成一行语法: @log 等价于 `func = log(func)`。本质很简单——函数是对象,装饰器是"接收函数、返回新函数"的函数——威力来自它可以叠加、可以参数化、可以作用于类。

前置要求:第4章闭包与高阶函数、第9章异常与重试、第11章内省与元数据。

本章路线:第一阶段(1.5 小时)读 §12.1–§12.3,理解本质并写出第一个装饰器;第二阶段(1.5 小时)读 §12.4–§12.5,掌握内置宝库与类装饰器;第三阶段(2 小时)读 §12.6–§12.8,完成生产装饰器并做章末实验。

本章知识导图

- └─ 12.1 本质 —— 函数是对象, @ 就是赋值
- └─ 12.2 functools.wraps —— 保住函数元数据
- └─ 12.3 带参数装饰器 —— 三层嵌套的由来
- └─ 12.4 内置宝库 —— lru_cache / contextmanager
- └─ 12.5 类装饰器 —— 装饰类与装饰方法
- └─ 12.6 生产三件套 —— 计时 / 重试 / 缓存
- └─ 12.7 组合与顺序 —— 装饰器的叠加规则
- └─ 12.8 生产实践 —— 横切关注点的边界

§12.1 本质:函数是对象,@ 就是赋值

函数在 Python 里是一等公民:可以赋给变量、放进列表、作为参数传递。装饰器就是利用这一点——@deco 加在 def 前,等价于 `func = deco(func)`。手写一个"给函数打日志"的装饰器,过程一目了然:

```
import time

def logged(func):
    """装饰器:打印函数名与耗时,然后调用原函数"""
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时 "
              f"{{(time.perf_counter() - start) * 1000:.2f}} ms")
        return result
    return wrapper

@logged
def fetch_data(url: str) -> str:
    return f"数据来自 {url}"

print(fetch_data("https://example.com"))
# fetch_data 耗时 0.01 ms
# 数据来自 https://example.com
```

C(函数指针包装)	Python(装饰器)
包装函数要手写 wrapper 并手动覆盖	@deco 一行完成"函数 = 包装(函数)"
宏包装有类型与作用域限制	装饰器是普通函数,任意函数/方法/类通用
包装链要逐层手动调用	装饰器可叠加,顺序即层级
包装后原符号丢失,难还原	保留原函数引用,随时可还原(f.__wrapped__)

wrapper 用 `*args`, `**kwargs` 透传一切参数,这是装饰器的通用骨架:外面包一层,里面调原函数,前后插入横切逻辑。注意原函数在 `def` 时就执行了包装——装饰器代码只运行一次(导入期),wrapper 才在每次调用时运行。

§12.2 functools.wraps:保住函数元数据

上面的写法有个问题:装饰后 `fetch_data.__name__` 变成了 "wrapper",help/签名/文档全丢,调试与文档全部失真。functools.wraps 把原函数的元数据复制到 wrapper 上,一行修复:

```

from functools import wraps

def logged(func):
    @wraps(func)                # 复制 __name__ / __doc__ / __wrapped__ 等
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时 "
              f"{{(time.perf_counter() - start) * 1000:.2f}} ms")
        return result
    return wrapper

@logged
def fetch_data(url: str) -> str:
    """按 URL 拉取数据"""
    return f"数据来自 {url}"

print(fetch_data.__name__)      # fetch_data(而不是 wrapper)
print(fetch_data.__doc__)      # 按 URL 拉取数据
help(fetch_data)               # 签名、文档全部正常

```

误区:装饰器不写 @wraps。 不带 wraps 的装饰器会让被装饰函数的名字、文档、签名全部变成 wrapper 的——堆栈跟踪里全是 wrapper, help() 显示错误文档, 依赖签名内省的工具(第11章的 inspect.signature、参数校验框架)全部失灵。生产规范:自定义装饰器一律第一行 @wraps(func), 这是装饰器的"安全带"。

§12.3 带参数装饰器:三层嵌套的由来

@retry(times=3) 这类"带参数的装饰器"比无参版多一层:装饰器工厂(接收配置)返回真正的装饰器(接收函数)返回 wrapper。三层嵌套是必然结构,别嫌绕,理解层级即可:

```

import time
from functools import wraps

def retry(times: int = 3, delay: float = 0.5):
    """装饰器工厂:返回真正装饰器,失败重试 times 次"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last = None
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last = e
                    time.sleep(delay * (attempt + 1))
            raise last # 全部失败,抛最后一次异常
        return wrapper
    return decorator

@retry(times=3, delay=0.2) # 工厂 → 装饰器 → wrapper
def unstable_api() -> str:
    ...

```

记忆方法:配置层(工厂)→ 函数层(装饰器)→ 调用层(wrapper)。如果只想记一个,记住"最外层拿参数,中间层拿函数,最内层拿调用"。带参数装饰器比无参版更常见,因为生产配置(次数、超时、白名单)总要参数化。

§12.4 内置宝库:lru_cache 与 contextmanager

标准库已经给你写好了一批生产级装饰器,先认识最常用的两个: `functools.lru_cache` 给纯函数加内存缓存——同样的入参直接返回上次结果,递归与频繁重复计算场景立省一个量级:

```

from functools import lru_cache

@lru_cache(maxsize=128) # 缓存最近 128 个入参的结果
def fib(n: int) -> int:
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(50)) # 瞬间返回,不再指数爆炸
print(fib.cache_info()) # CacheInfo(hits=..., misses=...)
fib.cache_clear() # 清空缓存

```

`@contextmanager` 把生成器变成上下文管理器——第5章手写的类版,这里一行搞定,是"with 逻辑"的标准生产写法:

```

from contextlib import contextmanager

@contextmanager
def timer(label: str):
    """用生成器实现上下文管理器:yield 前后是进入/退出逻辑"""
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"[{label}] 耗时 {(time.perf_counter() - start) * 1000:.2f} ms")

with timer("查询订单"):
    for _ in range(100000):
        pass

```

误区:给有副作用或可变入参的函数加 lru_cache。lru_cache 靠入参哈希做缓存键,两个陷阱:① 入参含 list/dict 会直接 TypeError(unhashable);② 函数有副作用(打印、改全局、读时间)时,缓存会跳过副作用——结果对而行为错。纪律:lru_cache 只用于纯函数;需要复杂键用 cache_key 函数转成可哈希元组。

§12.5 类装饰器:装饰类与装饰方法

装饰器同样作用于类:接收类,返回类(或包装对象)。经典场景是"给类的所有方法统一加日志/鉴权",配合第10章的元编程思路:

```

from functools import wraps

def log_methods(cls):
    """类装饰器:给类中所有公开方法包上调用日志"""
    for name, value in list(vars(cls).items()):
        if callable(value) and not name.startswith("__"):
            @wraps(value)
            def wrapped(*args, _f=value, _n=name, **kwargs):
                print(f"调用 {cls.__name__}.{_n}")
                return _f(*args, **kwargs)
            setattr(cls, name, wrapped)
    return cls

@log_methods
class Service:
    def start(self) -> None:
        print("启动服务")

    def stop(self) -> None:
        print("停止服务")

Service().start() # 调用 Service.start / 启动服务

```

细节:循环里用 `_f=value`, `_n=name` 默认参数绑定当前值——否则闭包捕获的是循环变量,所有方法都调用最后一个(第4章晚期绑定坑的实战版)。类装饰器与元类能力重叠,但更直白;规则照旧:简单场景用类装饰器,需要拦截"创建过程"才用元类。

§12.6 生产三件套:计时、重试、缓存

把本章知识组装成生产可用的三个装饰器。先看计时与重试的完整版(支持 `async` 函数用 `asyncio` 版,这里展示同步版):

```
import functools
import time

def timed(func):
    """计时装饰器:记录耗时,支持自定义日志函数"""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            cost = (time.perf_counter() - start) * 1000
            print(f"{func.__name__} 耗时 {cost:.2f} ms")
    return wrapper

def retry(times=3, delay=0.5, on=(Exception,)):
    """重试装饰器:只重试 on 指定的异常,指数退避"""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except on as e:
                    if attempt == times - 1:
                        raise
                    time.sleep(delay * (2 ** attempt))
            return wrapper
        return decorator
```

重试装饰器的生产要点: `on=(Exception,)` 让调用方显式声明"哪些异常值得重试"——默认全捕获是危险的(业务错误重试3次纯属浪费);退避用 `delay * 2**attempt` 指数增长,避免重试风暴。`timed` 用 `finally` 保证异常路径也计时——这是"计时必须覆盖失败路径"的细节。

再看缓存装饰器——缓存键的设计是灵魂:默认用 `repr` 元组,可变参数先转成可哈希形式:

```

import functools

def memoize(func):
    """手写缓存:入参转可哈希键,命中直接返回"""
    cache: dict = {}
    MISS = object()

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        try:
            key = (args, tuple(sorted(kwargs.items())))
        except TypeError:
            return func(*args, **kwargs)  # 参数不可哈希,跳过缓存
        result = cache.get(key, MISS)
        if result is MISS:
            result = func(*args, **kwargs)
            cache[key] = result
        return result
    return wrapper

@memoize
def query(user_id: int) -> dict:
    """模拟查询:同一用户只查一次"""
    return {"id": user_id, "name": "user" + str(user_id)}

print(query(1) is query(1))          # True — 第二次命中缓存

```

与 `lru_cache` 相比,手写版展示了缓存三要素:键构造、命中检查、存储。生产直接用 `lru_cache`;手写它的意义在于理解——当需要"按自定义键缓存""分环境缓存""缓存失效策略"时,你会知道自己要改哪里。

§12.7 组合与顺序:装饰器的叠加规则

多个装饰器叠加时,执行顺序是从下往上(离函数最近的先包装)。看例子:

```

@timed                                # 外层:最后包装,最先执行外层逻辑
@retry(times=2, delay=0.1)
def fragile_call() -> str:
    import random
    if random.random() < 0.5:
        raise ConnectionError("偶发失败")
    return "成功"

print(fragile_call())
# 调用链:timed.wrapper -> retry.wrapper -> fragile_call
# 重试发生在 timed 计时内部,一次计时的总耗时包含所有重试

```

误区:装饰器顺序想当然。顺序错了行为就错:把 `retry` 放外层、`timed` 放内层,则每次重试都被单独计时,总耗时被拆成多段;把缓存放外层、校验放内层,则未校验的数据可能已进缓存。规则:横切关注点按"越通用越在外"排列(缓存在最外、重试次之、业务校验最内),并对照调用链验证。

§12.8 生产实践:横切关注点的边界

装饰器是横切关注点(日志、鉴权、限流、缓存、重试)的标准载体。判断"该不该写成装饰器"的三个标准:① 多函数共享(两三个函数用就别抽象);② 行为正交(与业务逻辑无关,可独立测试);③ 参数化需求明确(次数、白名单、阈值)。

C(手工横切)	Python(装饰器)
每个函数重复写超时/重试样板	@retry(3) 一行,配置显式
横切代码与业务代码交织,改一处动全身	横切逻辑独立成装饰器,业务函数干净
加一个横切点要动 N 个函数	加一个 @ 符号即可,回滚同样一行
横切逻辑难以单独测试	装饰器可独立单测(配假函数)

工程建议 1:装饰器库单独放一个模块(decorators.py),每个装饰器配单元测试:用假函数验证包装行为、异常路径、参数透传;文档里写清"配置含义、适用场景、顺序要求"——装饰器是隐式逻辑,文档就是它的安全网。

工程建议 2:装饰器里不要吞异常、不要改签名、不要改返回值类型——它应该"透明"。排障时用 functools 的 `__wrapped__` 链(`fragile_call.__wrapped__`)直达原函数,这条链要保留,别在装饰器里覆盖它。

本章速查表

场景	写法
无参装饰器	<code>def deco(func): @wraps(func) def w(*a, **k): ...; return w</code>
带参装饰器	<code>def deco(conf): def d(func): ...; return d</code> 三层嵌套
保元数据	wrapper 第一行 <code>@functools.wraps(func)</code>
函数缓存	<code>@lru_cache(maxsize=128)</code> ,仅纯函数
清缓存	<code>fib.cache_clear()</code> ,查看 <code>fib.cache_info()</code>
生成器上下文	<code>@contextmanager</code> + <code>yield</code> 前后
类统一包装	类装饰器遍历 <code>vars(cls)</code> ,绑定默认参防晚期绑定
重试配置	<code>on=(异常,)</code> 显式声明,指数退避
计时含异常	<code>finally</code> 里算耗时
缓存键	<code>args + sorted(kwargs)</code> 转元组;不可哈希则跳过缓存
叠加顺序	从下往上包装,越通用越在外
还原原函数	<code>func.__wrapped__</code> (wraps 自动设置)
透明原则	不吞异常、不改签名、不改返回类型
使用标准	多函数共享 + 行为正交 + 参数化
测试	配假函数独立单测,异常路径必测
文档	写清配置含义、适用场景、顺序要求

最小必会清单

1. 能手写无参装饰器骨架,并解释 `@deco` 与 `func = deco(func)` 的等价关系
2. 能说出 `wraps` 的作用与缺失后果,并始终在装饰器里使用它
3. 能写出带参数装饰器(工厂→装饰器→wrapper 三层),并配置重试/超时
4. 能说出 `lru_cache` 的适用条件(纯函数)与两个陷阱(不可哈希、副作用)
5. 能用 `@contextmanager` 把生成器变成上下文管理器
6. 能解释装饰器叠加顺序,并用调用链验证行为

自测题

Q

Q1. `@deco` 到底做了什么?装饰器代码什么时候运行,wrapper 什么时候运行?

答案:`@deco` 等价于 `func = deco(func):deco` 在定义语句执行时(导入期)被调用一次,接收原函数、返回 wrapper,名字 `func` 绑定到 `wrapper`。之后每次调用 `func()` 实际执行 `wrapper`。所以装饰器工厂逻辑在导入期运行一次,包装逻辑每次调用运行——用这个区分,就能判断装饰器里哪些代码可以放函数外(初始化缓存、编译正则)。

Q

Q2. 不带 `@wraps` 的装饰器会带来哪些具体问题?

答案:① `__name__` 变成 `wrapper`,堆栈跟踪与日志里全是 `wrapper` 层,定位困难;② `__doc__` 丢失,`help()` 与文档生成器显示错误内容;③ `inspect.signature` 拿到 `wrapper` 的 `(*args, **kwargs)`,依赖签名的框架(参数校验、自动文档)失灵;④ `__wrapped__` 链不存在,无法还原原函数。`@wraps` 一行全部修复。

Q

Q3. 为什么带参数装饰器需要三层?能少一层吗?

答案:因为 `@retry(times=3)` 的求值过程:`retry(times=3)` 先执行返回装饰器,装饰器再接收函数返回 `wrapper`。三层分别对应:配置参数层(`retry`)、函数层(`decorator`)、调用层(`wrapper`)。无参装饰器两层即可(装饰器+`wrapper`);带参版少一层就没地方放配置。要一眼看懂,记住"最外层拿参数、中间拿函数、最内拿调用"。

Q

Q4. `lru_cache` 装饰一个"打印日志并返回当前时间"的函数,为什么结果可疑?

答案:该函数有副作用(打印)且结果依赖时间(非纯函数)。`lru_cache` 命中缓存时直接返回上次结果,不执行函数体——副作用被跳过、时间结果过时。缓存只适合纯函数:同样的入参永远产生同样的结果、无外部影响。非纯函数要么不用缓存,要么把"时间""环境"也纳入缓存键。

Q

Q5. 类装饰器循环里为什么用 `_f=value`, `_n=name` 默认参数?

答案:闭包捕获的是变量而非值:循环结束后 `value/name` 指向最后一个方法,所有 `wrapper` 实际都调用它(第4章晚期绑定坑)。默认参数在定义时求值,把当前迭代的值"钉"进 `wrapper`,每个 `wrapper` 调用各自的方法。这是装饰器与循

环组合的头号陷阱,任何“循环里造闭包”的场景都适用。



Q6. @timed 与 @retry 叠加时,顺序不同会怎样?如何决定顺序?

答案:@timed 在上、@retry 在下:timed 包住 retry,一次计时包含所有重试(总耗时)。反过来则每次重试被单独计时。规则:通用性高的在外——缓存最外(命中就不进内层)、重试其次、业务逻辑最内;验证方式:从内到外列出调用链,检查每个横切点是否在期望的边界上执行。

章末实验题:装饰器工具箱

实验 12•1 计时与重试装饰器库(覆盖:\$12.1–\$12.6)

需求:① 实现 timed 装饰器:支持自定义日志函数(默认 print),记录名称、耗时(ms)、成功/失败标记;② 实现 retry 装饰器:支持次数、退避基数、可重试异常元组、最大总时长上限(超限立即放弃);③ 实现 cached 装饰器:支持 TTL 过期时间(到期的键自动失效重算);④ 三个装饰器可叠加,顺序按"cached 最外、retry 次之、timed 最内";⑤ 用一个模拟的不稳定服务(按概率失败 + 随机耗时)验证:重试后成功、缓存命中不再调用、计时包含全部重试;⑥ 输出验证报告并写单元测试(用假函数断言调用次数)。

覆盖知识点:装饰器本质与 wraps(\$12.1、\$12.2)/带参数装饰器(\$12.3)/缓存键与 TTL(\$12.6)/叠加顺序(\$12.7)。

实验环境:Python 3.10+,单文件 `decorators_lab.py`。

```

# decorators_lab.py — 生产装饰器工具箱(覆盖第12章全部知识点)
import functools
import random
import sys
import time

def timed(logger=print):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            start = time.perf_counter()
            try:
                return func(*args, **kwargs)
            finally:
                ok = sys.exc_info()[0] is None
                logger(f"[timed] {func.__name__} "
                    f"{{(time.perf_counter() - start) * 1000:.2f}} ms({{'成功' if ok else '异常'}})")
        return wrapper
    return decorator

def retry(times=3, backoff=0.2, on=(Exception,), max_total=10.0):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            deadline = time.monotonic() + max_total
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except on:
                    if attempt == times - 1:
                        raise
                    if time.monotonic() >= deadline:
                        raise
                    time.sleep(backoff * (2 ** attempt))
            return wrapper
    return decorator

def cached(ttl: float = 60.0):
    def decorator(func):
        cache: dict = {}
        store = {"hits": 0, "misses": 0}
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            try:
                key = (args, tuple(sorted(kwargs.items())))
            except TypeError:
                return func(*args, **kwargs)
            entry = cache.get(key)
            if entry and time.monotonic() - entry[1] < ttl:
                store["hits"] += 1
                return entry[0]
            store["misses"] += 1
            result = func(*args, **kwargs)

```

```

        cache[key] = (result, time.monotonic())
        return result
    wrapper.stats = lambda: dict(store)
    return wrapper
return decorator

```

```

@cached(ttl=1.0)
@retry(times=3, backoff=0.1)
@timed()
def unstable_service(key: str) -> str:
    """模拟不稳定服务:30% 概率失败,成功返回带时间戳结果"""
    if random.random() < 0.3:
        raise ConnectionError("服务偶发不可用")
    time.sleep(random.uniform(0.01, 0.05))
    return f"{key}-{time.time():.3f}"

def main() -> None:
    random.seed(7)
    print("== 首次调用(可能触发重试)==")
    r1 = unstable_service("order")
    print("结果:", r1)

    print("== 缓存命中(1 秒内再调,不执行原函数)==")
    for _ in range(3):
        r2 = unstable_service("order")
        assert r2 == r1, "缓存未命中,结果不一致!"
    print("命中统计:", unstable_service.stats())

    print("== TTL 过期后重算 ==")
    time.sleep(1.2)
    r3 = unstable_service("order")
    print("过期后结果:", r3, "(应不同于首次)")

    print("== 断言与验证 ==")
    assert unstable_service.stats()["hits"] >= 3
    assert unstable_service.stats()["misses"] >= 2
    print("全部验证通过")

if __name__ == "__main__":
    main()

```

设计说明:① 装饰顺序 cached 最外:缓存命中时重试与计时都不执行,行为正确;② retry 的 max_total 用 time.monotonic()(不受系统时间调整影响)与"预计退避是否超限"双保险,绝不无限拖延;③ cached 用 (args, sorted(kwargs)) 做键、monotonic 时间戳做 TTL,miss/hit 统计挂在 wrapper 上便于断言;④ 实验用 assert 自动验证,加随机种子保证可复现;⑤ 三个装饰器都可独立单测——配一个假函数断言调用次数与参数透传即可。

下一章预告:第13章 模块与工程化——import 机制、sys.path 与 __init__.py、venv 与 pip、pyproject.toml、项目布局与发布。让代码从"能跑"走向"能交付"。